Debounce and Throttle

Imagine you run a food delivery startup.

Customers keep spamming the “Order Now” button.

Two ways to handle this:

They are rate-limiting strategies for high-frequency events.

- **Debounce = Wait until user stops**

You wait 2 seconds after the last click before placing the order.

If they keep clicking, timer resets.

👉 **Only final click counts.**

Used in:

- **Search bars**
- **Auto-save**
- **Resize events**
-

- **Throttle = Limit frequency**

No matter how many clicks happen, you allow only 1 order per 2 seconds.

Used in:

- **Scroll events**
- **Mouse movement tracking**
- **Rate limiting APIs**

The Deeper Why

Browsers fire events VERY FAST.

- **Scroll → 60 times per second**

- **Keypress → every keystroke**

Without control:

- **UI lag**
- **Memory bloat**
- **API overload**
- **Poor UX**

This is performance engineering.

- **Debounce**

A higher-order function that delays execution until a pause occurs.

```
function debounce(fn,delay){
   let timer;

   return function(...args){
      clearTimeout(timer);// clear timeout on every function call

      timer=setTimeout(()=>{
         fn(...args);// run function after delat
      },delay);
   }
}

const sayHello=debounce((name)=>{
   console.log("Hello",name);
```

```
},2000);


for(let i=0;i<100; i++){
   sayHello("Prajwal");// evben though we call it 100 times but it only run once when calling interval is 2000ms

}
```

- **Throttle**

Executes function at fixed intervals.

```
function throttle(fn,delay){
   let isThrottled=false;

   return function(...args){
      if(isThrottled) return;

      fn(...args);

      isThrottled=true;

      setTimeout(()=>{
         isThrottled=false;
      },delay);
   }
}
```

```
const log=throttle((name)=>{
   console.log("Throttled ",name);
},2000);

setInterval(()=>{
   log("prajwal")
},300);// even though interval is running every 300ms clg is happening at 2000ms beacuse of throttle .
```

Interview Nuance

❓ **Why use apply(this, args)?**

To preserve:

- **Execution context**
- **Arguments**

Q1: Difference between debounce and throttle?

Answer:
Debounce delays execution until activity stops.
Throttle limits execution frequency.

Q2: When NOT to use debounce?

If you need continuous updates (like progress bars).

REAL WORLD USE CASES — DEBOUNCE

1️⃣ 🔎 Google-Style Search Autocomplete

You only care about the **final intent**, not intermediate letters.

2️⃣ 📧 Email Availability Check (Signup Forms)

You **don’t** want to hit DB for every keystroke.

💾 Auto-Save Draft (Medium, Notion, Gmail)

You type continuously → auto-save triggers after you pause.

🚀 REAL WORLD USE CASES — THROTTLE

1️⃣ 📜 Infinite Scroll Feed

You want regular position updates — but not 100 times/sec.

2️⃣ 📊 Scroll Progress Indicator

Like reading progress bar on blogs.

3️⃣ 📈 Trading Dashboard

If you build:

- Real-time crypto prices
- Order book visualizer

WebSocket may push 50 updates/sec.

You throttle:

- UI re-renders
- Chart updates

Why?
Rendering is expensive.
Human eye doesn’t need 50fps data updates.

Throttle to 10fps → smooth + efficient.

Why use apply..?

When you do this:

obj.method()

Inside method, this = obj.

But if you pass it like this:

debounce(obj.method, 500)

Now this is LOST.

That’s the real-world bug.

🧠 **Two Ways to Preserve this**

✅ **1. Use apply() inside wrapper**

✅ **2. Use .bind() when passing function**

Both are valid.

```
const user = {
name: "Prajwal",
greet(message) {
console.log(message, this.name);
}
};
```

✅ 1️⃣ **Preserve this Using apply() Inside Debounce**

Here, we fix this inside the wrapper itself.

Debounce with apply

```
function debounce(fn, delay) {
  let timer;

  return function (...args) {
    const context = this; // capture this
```

```
    clearTimeout(timer);

    timer = setTimeout(() => {
      fn.apply(context, args); // preserve this
    }, delay);
  };
}


user.debouncedGreet = debounce(user.greet, 1000);

user.debouncedGreet("Hello");
```

2. with Bind

```
const fn = debounce(obj.method.bind(obj), 500);


const debouncedGreet=debounce(user.greet.bind(user),500);
```